Bitácora S09: Mexi Banco

Que el aviso no tumbe la transferencia

Sistemas Distribuidos (Semestre 2027-1, FI-UNAM)

Victor Emiliano Calderón Gutiérrez

No. de cuenta 423097331

24 de octubre de 2026

Entorno

Hora: 00:52

Para este laboratorio partí de la rama `05-mensajeria-base` del repositorio https://github.com/OscarRuiz21/mexi-banco, que es la solución del lab S08 (la saga con su circuit breaker) más un RabbitMQ ya instalado en el `docker-compose.yml` pero que todavía nadie usa. La cloné dentro de mi carpeta de entregas como `s09/mexi-banco`, creé mi rama de trabajo `trabajo-s09` y, al igual que en los labs anteriores, desactivé `core.autocrlf` en el clon para que `mvnw` no se rompiera con los finales de línea de Windows. Todo lo corrí en Windows 11 con Docker Desktop y Git Bash.

Antes de levantar nada volví a cambiar el nombre del proyecto de Compose: la rama base lo llama `mexi-banco-v06`, que es el mismo del lab S07, y con ese nombre iba a reusar su volumen de Postgres (donde Ana y Beto ya tienen otros saldos). Por lo que lo dejé como `mexi-banco-s09` (y la red como `mexibanco-s09`). Además, como la guía lo permite para no cargar tanto la laptop, levanté una sola copia de `cuenta` y no levanté `spei`, que este lab no usa:

```
docker compose build discovery gateway cuenta movimiento transferencia notificacion
docker compose up -d --scale cuenta=1 db rabbitmq discovery gateway cuenta movimiento transferencia
    notificacion
```

Esta vez las imágenes se construyeron sin problemas, pero me tocó un error distinto: unos tres minutos después de arrancar, el contenedor de `discovery` se quedó congelado (dejó de escribir en su log y de contestar cualquier petición HTTP, incluido su health check), y Docker ni siquiera lo pudo detener (`tried to kill container, but did not receive an exit event`). Me di cuenta porque al querer volver a encender `notificacion` al final de la Parte 0, Compose se negó, ya que `notificacion` espera a que `discovery` esté *healthy*. Lo resolví reiniciando el motor de Docker (`docker desktop restart`), quitando el contenedor trabado y volviendo a levantar todo; los datos no se perdieron, porque Postgres vive en un volumen.

1. El problema

Hora: 01:05

Tal como lo pide la guía, antes de tocar el código reproduje el problema con la versión base. Primero abrí el panel de RabbitMQ para ver cómo estaba el broker antes de usarlo: en *Exchanges* solo aparecen los que trae RabbitMQ (los `amq.*` y el de omisión) y en *Queues and Streams* no hay ninguna cola, es decir, nadie ha declarado nada todavía.

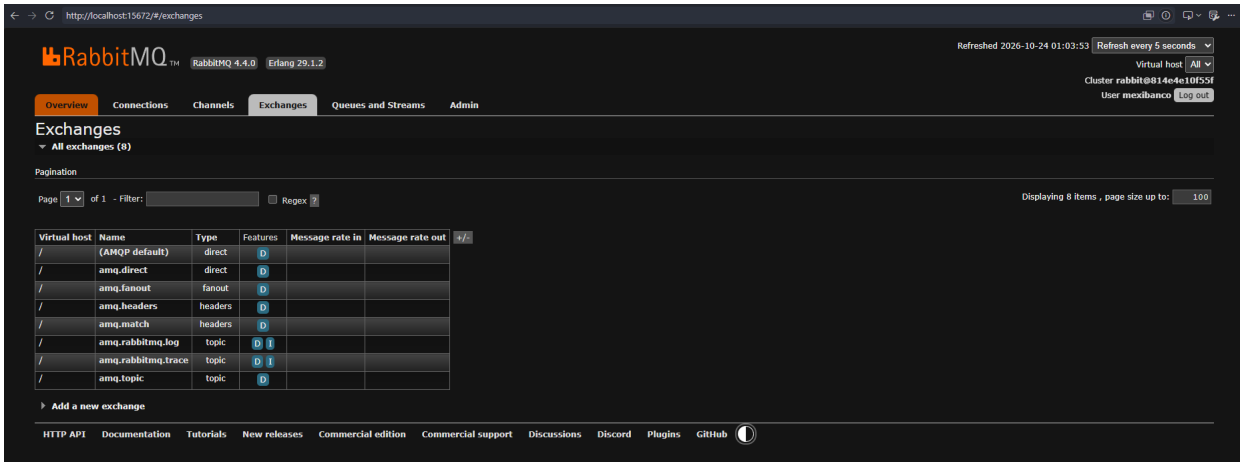


Figura 1: El panel de RabbitMQ antes del lab: solo los exchanges que trae el broker.

Después di de alta a Ana (`00218000000000001`) con 1000 y a Beto (`00218000000000002`) con 500 por el gateway, apagué `notificacion` con `docker compose stop notificacion` y mandé una transferencia de 50 de Ana a Beto.

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s09/mexi-banco (trabajo-s09)
$ curl -s -w ' [HTTP %{http_code}]\n' -X POST $API/cuentas -H 'Content-Type: application/json' -d "{\"clabe\":\"$ANA\",\"titular\":\"Ana\",\"saldoInicial\":1000}"
{"id":1,"clabe":"00218000000000001","titular":"Ana","saldo":1000} [HTTP 201]

battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s09/mexi-banco (trabajo-s09)
$ curl -s -w ' [HTTP %{http_code}]\n' -X POST $API/cuentas -H 'Content-Type: application/json' -d "{\"clabe\":\"$BETO\",\"titular\":\"Beto\",\"saldoInicial\":500}"
{"id":2,"clabe":"00218000000000002","titular":"Beto","saldo":500} [HTTP 201]
```

Figura 2: Alta de las cuentas de Ana y Beto: las dos responden 201.

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s09/mexi-banco (trabajo-s09)
$ docker compose stop notificacion
[+] stop 1/1
 ✓Container mexi-banco-s09-notificacion-1 Stopped

battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s09/mexi-banco (trabajo-s09)
$ curl -i -X POST $API/transferencias -H 'Content-Type: application/json' -d "{\"claveOrigen\":\"$ANA\",\"claveDestino\":\"$BETO\",\"monto\":50}"
HTTP/1.1 503 Service Unavailable
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Sat, 24 Oct 2026 07:05:50 GMT

{"type":"about:blank","title":"Service Unavailable","status":503,"detail":"No se pudo contactar a otro servicio: I/O error on POST request for \"http://notificacion/notificaciones\": Connect timed out","instance":"/transferencias"}
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s09/mexi-banco (trabajo-s09)
```

Figura 3: Con `notificacion` apagado, la transferencia responde `503 Service Unavailable`: `Connect timed out` al llamar a `http://notificacion/notificaciones`.

El cliente recibió un `503` que dice que no se pudo contactar a otro servicio, ya que la transferencia intentó hacer `POST` a `http://notificacion/notificaciones` y la conexión se agotó (como la mandé en cuanto apagué el servicio, el directorio todavía lo anunciaba y el balanceador intentó llegar a él). Sin embargo, al consultar la transferencia y los saldos se ve que el dinero sí se movió:

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s09/mexi-banco (trabajo-s09)
$ curl -s "$API/transferencias?estado=ABONADA"; echo
[{"id":1,"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":50.00,"fecha":"2026-10-24T07:05:49.275757Z","estado":"ABONADA","motivo":null,"saldoOrigenTrasCargo":950.00,"intentosDeCompensacion":0,"actualizada":"2026-10-24T07:05:49.35039
3Z"}]

battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s09/mexi-banco (trabajo-s09)
$ curl -s $API/cuentas/$ANA; echo
{"id":1,"clabe":"00218000000000001","titular":"Ana","saldo":950.00}

battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s09/mexi-banco (trabajo-s09)
$ curl -s $API/cuentas/$BETO; echo
{"id":2,"clabe":"00218000000000002","titular":"Beto","saldo":550.00}
```

Figura 4: La transferencia se quedó en `ABONADA` y los saldos ya se movieron: Ana 950 y Beto 550.

Es decir, Ana pasó de 1000 a 950 y Beto de 500 a 550, por lo que el cargo, el abono y los asientos ya ocurrieron, pero la transferencia se quedó en `ABONADA` y nadie la va a terminar, ni siquiera al volver a encender `notificacion`.

¿Por qué pasó? Porque al final de la saga la transferencia se quedaba esperando la respuesta HTTP de `notificacion` antes de marcarse `COMPLETADA`, entonces, con `notificacion` apagado, la excepción de esa llamada (un aviso, que bien podía esperar) salió hasta el cliente y dejó a medias una operación cuyo dinero ya había llegado.

2. Publicar el evento

Hora: 01:17

Tal como lo vimos en la clase, la salida es poner un broker en medio: en lugar de "llamar por teléfono" a `notificacion` y quedarse esperando a que conteste, la transferencia deja un recado en un buzón (un exchange de RabbitMQ) y sigue con lo suyo. Todos los cambios de esta parte están en `transferencia` y los marqué en el código con comentarios "Lab S09". Los pedazos importantes son los siguientes.

2.1 · El evento y su `eventoId`

Lo primero fue que cada transferencia naciera con su `eventoId`. Lo importante aquí es que se genera junto con la fila (en el constructor de `Transferencia`) y se guarda en la tabla, y no al momento de publicar, ya que si se generara al publicar, cada reintento parecería un evento nuevo y el consumidor no tendría forma de reconocer un duplicado:

```
@Column(unique = true)
private UUID eventoId;
...
this.eventoId = UUID.randomUUID();   // en el constructor, junto con la fecha y el estado INICIADA
```

El evento en sí es un `record` propio (`mensajeria/EventoTransferencia`) con los campos del contrato, y no la entidad `Transferencia`, porque es lo que va a ver quien escuche, y cambiar una columna de la tabla no debería romper a nadie afuera. Tiene dos fábricas, `realizada(...)` y `revertida(...)` (la revertida sin `saldoDestino`, ya que a destino nunca se le tocó), y un método `routingKey()` que regresa `transferencia.realizada` o `transferencia.revertida` según el tipo.

2.2 · El exchange y el publicador

Del lado de `transferencia` lo único que se declara en el broker es el exchange, ya que la transferencia publica ahí y no sabe (ni le importa) qué colas hay detrás: las colas son de quien consume. En `ConfiguracionMensajeria` quedó el exchange `transferencias` de tipo *topic* y durable, y un convertidor JSON que usa el mismo `ObjectMapper` de Spring Boot (para que la fecha salga como texto ISO-8601 y no como número):

```
@Bean
TopicExchange exchangeTransferencias() {
    return ExchangeBuilder.topicExchange("transferencias").durable(true).build();
}
```

Después, en `application.yml`, activé los *publisher confirms*, los *returns* y un *timeout* de conexión corto:

```
rabbitmq:
  connection-timeout: 2s              # si el broker no contesta, rendirse a los 2 s
```

```
  publisher-confirm-type: correlated  # el broker contesta ack o nack por cada mensaje
  publisher-returns: true             # y regresa el que ninguna cola recibió...
  template:
    mandatory: true                   # ...en lugar de tirarlo en silencio
```

Y el que publica es `PublicadorDeEventos`. Lo que hace es mandar el evento con una `CorrelationData` (que es lo que une la confirmación, que llega después y por otro hilo, con este mensaje), marcarlo como persistente (para que se escriba a disco) y después *esperar* hasta 5 segundos la respuesta del broker. Solo regresa normal si el broker dijo `ack` y además ninguna cola lo regresó; en cualquier otro caso lanza `EventoNoPublicadoException`:

```
CorrelationData correlacion = new CorrelationData(id);
rabbit.convertAndSend("transferencias", evento.routingKey(), evento, mensaje -> {
    mensaje.getMessageProperties().setDeliveryMode(MessageDeliveryMode.PERSISTENT); // a disco
    mensaje.getMessageProperties().setMessageId(id);
    return mensaje;
}, correlacion);
CorrelationData.Confirm confirmacion = correlacion.getFuture().get(esperaConfirmacionMs, MILLISECONDS);
if (!confirmacion.isAck())               -> "El broker rechazo el evento (nack)"
if (correlacion.getReturned() != null)   -> "Ninguna cola recibio el evento"
```

2.3 · La saga publica en lugar de llamar

En `TransferenciaService` cambié la dependencia de `NotificacionCliente` por el publicador, y al final de `transferir` quité las dos llamadas `notificaciones.notificar(...)`. Ahora la saga primero guarda `COMPLETADA` y después publica:

```
t.marcarCompletada();
t = guardar(t);                                    // primero la base dice COMPLETADA
avisar(EventoTransferencia.realizada(t.getEventoId(), t.getId(), claveOrigen, claveDestino, monto,
    t.getSaldoOrigenTrasCargo(), destino.saldo(), t.getActualizada()));   // luego el recado
```

El orden importa, ya que al revés `notificacion` podría avisarle al cliente de una transferencia que la base todavía no da por hecha. Lo mismo hice en `compensar`: cuando queda `COMPENSADA` se publica `TransferenciaRevertida` (y como el reintento programado de la S08 pasa por ese mismo método, también avisa cuando él la termina); si la devolución también falla y queda `COMPENSACION_PENDIENTE`, no se publica nada porque el dinero todavía no regresa. Por último, el método `avisar` atrapa cualquier error al publicar y solo lo escribe en el log, es decir, si el broker no confirma, la transferencia *no* se revierte, ya que el dinero se movió bien y lo único que se pierde es el aviso. Ya sin uso, borré `clientes/NotificacionCliente.java` y la variable `NOTIFICACION_URL` de `transferencia` en el compose (`spei` tiene su propia copia y sigue con HTTP).

Como `TransferenciaServiceTest` usaba `NotificacionCliente`, también hice el ajuste que indica la guía para que la entrega quede limpia: el doble ahora es del publicador, el camino feliz verifica *un* evento en lugar de dos llamadas, y la prueba de la compensada ya no exige cero interacciones, sino que se publique el aviso (la especificación cambió).

2.4 · Probarlo con `notificacion` apagado

Reconstruí solo `transferencia`, apagué `notificacion` y mandé otra transferencia de 50:

Esta vez la transferencia respondió `201` y quedó `COMPLETADA` aunque `notificacion` estaba apagado. Sin embargo, el log de `transferencia` dice qué pasó con el evento:

Y en el panel ya aparece el exchange `transferencias`, de tipo *topic* y durable, pero sin ninguna cola enlazada:



Figura 5: Con `notificacion` apagado, la transferencia 2 responde `201 Created`, queda `COMPLETADA` y ya trae su `eventoId`.

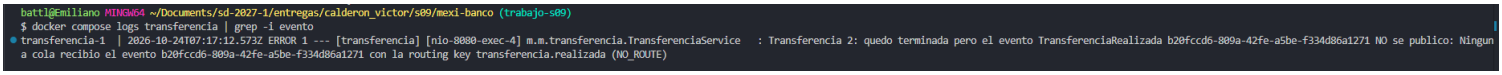


Figura 6: El log: la transferencia quedó terminada pero el evento no se publicó, ya que ninguna cola lo recibió (`NO_ROUTE`).

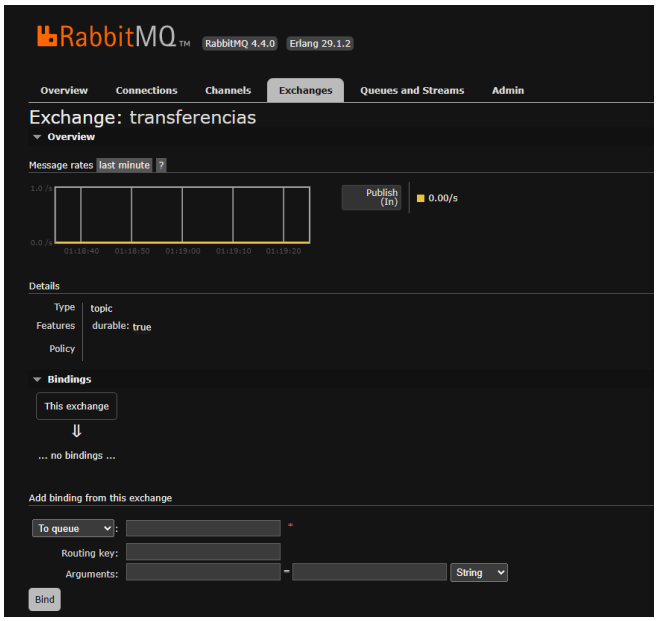


Figura 7: El exchange `transferencias` en el panel: *topic*, durable y sin *bindings*.

Es decir, ese aviso se perdió. Un exchange no guarda mensajes, solo decide a qué colas mandarlos, y como todavía nadie había declarado una cola enlazada a él, el broker no tenía a dónde ponerlo. Gracias a `mandatory` por lo menos no se tiró en silencio (el broker lo regresó y quedó en el log), pero nadie lo vuelve a intentar. Esto se arregla en la siguiente parte, cuando `notificacion` declara su cola.

2.5 · ¿Qué me garantiza el *publisher confirm*?

La confirmación me garantiza que el broker *ya tiene* el mensaje: cuando contesta `ack`, RabbitMQ lo recibió y (como es persistente y la cola es durable) lo escribió a disco, por lo que ya no se pierde aunque el broker se reinicie; y junto con `mandatory`, también sé que por lo menos una cola lo recibió. Lo que *no* me garantiza es que alguien lo vaya a procesar, ni cuándo: con `notificacion` apagado el broker confirma igual (el mensaje se queda esperando en la cola), así que el confirm habla del broker, no del consumidor. Y tampoco me cubre lo que pasa antes de publicar: si el proceso de `transferencia` muere justo entre `guardar(t)` y `avisar(...)`, la base dice `COMPLETADA` y el evento nunca salió, y no

hay confirmación que esperar porque nunca se mandó nada. Ese hueco (la doble escritura entre la base y el broker) es el que resuelve el outbox del reto.

3. Desacople en tiempo

Hora: 01:24

Del otro lado del buzón, `notificacion` necesitaba tres cosas: su cola enlazada al exchange, un consumidor que leyera cada mensaje y guardara los avisos, y una regla, que es confirmar el mensaje solo *después* de guardar. Los cambios de esta parte están en `notificacion`.

3.1 · La cola y el consumidor

La cola es de `notificacion`, por eso se llama como su dueño, y la declaré en la clase `ConfiguracionMensajeria` (paquete `mensajeria`) junto con su *binding* al exchange con el patrón `transferencia.*` (es decir, le llegan tanto las realizadas como las revertidas). También declaré ahí el exchange `transferencias`, con las mismas propiedades que en `transferencia`, para poder enlazar la cola aunque `notificacion` arranque primero:

```
TopicExchange transferencias = ExchangeBuilder.topicExchange("transferencias").durable(true).build();
Queue cola = QueueBuilder.durable("notificacion.transferencias").build();
return new Declarables(transferencias, cola, BindingBuilder.bind(cola).to(transferencias).with("
    transferencia.*"));
```

Del lado de `notificacion` el evento es otra vez un `record` propio (`EventoTransferencia`, una copia local del contrato), igual que con HTTP, donde cada servicio tenía su DTO. En `NotificacionService` agregué `avisarDeTransferencia(evento)`, que en una sola transacción guarda *dos* avisos por `TransferenciaRealizada` (uno a origen como `TRANSFERENCIA_ENVIADA` y otro a destino como `TRANSFERENCIA_RECIBIDA`) y *uno* por `TransferenciaRevertida` (a origen, como `DEVOLUCION_TRANSFERENCIA`), con el mismo texto de siempre: tipo, monto y saldo resultante.

Para el consumidor, en `application.yml` cambié el modo de confirmación a manual (por omisión Spring hace el ack solo al regresar el método) y limité a 5 los mensajes sin confirmar que puede tener a la vez:

```
listener:
  simple:
    acknowledge-mode: manual   # el consumidor dice cuándo (basicAck)
    prefetch: 5                # máximo 5 sin ack a la vez (250 por omisión)
```

Y el consumidor (`ConsumidorDeTransferencias`) lee el JSON a mano, guarda los avisos (ahí se hace el commit) y solo después le dice al broker que ya lo tiene:

```
@RabbitListener(queues = "notificacion.transferencias")
public void recibir(Message mensaje, Channel canal) throws IOException {
    long etiqueta = mensaje.getMessageProperties().getDeliveryTag();
    EventoTransferencia evento = json.readValue(mensaje.getBody(), EventoTransferencia.class);
    List<Notificacion> avisos = notificaciones.avisarDeTransferencia(evento);   // aquí hace commit
    log.info("[consumidor] {} de la transferencia {} (evento {}, redelivered={}): {} aviso(s) nuevo(s)"
    , ...);
    canal.basicAck(etiqueta, false);   // DESPUÉS de guardar: "ya lo tengo, bórralo"
}
```

3.2 · El experimento

Al reconstruir y arrancar `notificacion`, se conectó al broker y declaró su cola, que quedó enlazada al exchange con `transferencia.*` y con un consumidor. Después lo apagué e hice tres transferencias, de 11, 12 y 13:

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s09/mexi-banco (trabajo-s09)
$ docker compose stop notificacion
[+] stop 1/1
 ✓ Container mexi-banco-s09-notificacion-1 Stopped

battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s09/mexi-banco (trabajo-s09)
$ for m in 11 12 13; do curl -s -o /dev/null -w "monto $m -> HTTP %{http_code}\n" -X POST $API/transferencias -H 'Content-Type: application/json' -d "{\"claveOrigen\":\"$ANA\",\"claveDestino\":\"$BETO\",\"monto\":$m}"; done
monto 11 -> HTTP 201
monto 12 -> HTTP 201
monto 13 -> HTTP 201

battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s09/mexi-banco (trabajo-s09)
$ docker compose exec -T rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged consumers
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged consumers
notificacion.transferencias     3       0       0
```

Figura 8: Con `notificacion` apagado, las tres transferencias responden `201` y la cola tiene 3 mensajes *ready* y 0 consumidores.

Las tres respondieron `201` y la cola se quedó con los tres mensajes esperando, lo cual también se ve en el panel:

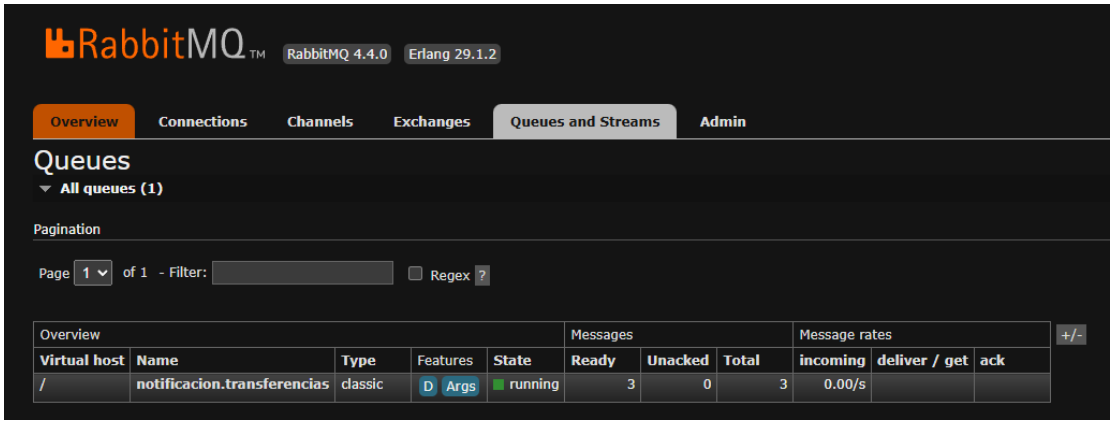


Figura 9: La cola `notificacion.transferencias` en el panel: durable, con 3 mensajes *Ready*.

Al volver a encender `notificacion`, la cola bajó a 0 en cuanto arrancó, en su log aparece una línea por cada evento (cada una con `redelivered=false` y 2 avisos nuevos) y las bandejas de Beto y de Ana ya tienen sus tres avisos:

Es decir, el productor y el consumidor ya no tienen que estar vivos al mismo tiempo, que es justo el desacople en tiempo que vimos en la clase: las tres transferencias se completaron sin que `notificacion` existiera en ese momento, y los recados lo esperaron en la cola hasta que volvió. Algo que también se nota es que la transferencia 2 (la de la Parte 1) no tiene aviso en ninguna bandeja, ya que en ese momento no había cola y ese mensaje se perdió.

3.3 · El aviso de la reversa

Por último, probé la reversa con la falla provocada de la S08: levanté `transferencia` con `FALLAR_ABONO_A_CLABE` apuntando a Beto y mandé una transferencia de 150:

La saga devolvió el cargo (por eso respondió `502` con `COMPENSADA` y el saldo de Ana quedó en 864, igual que antes de esta transferencia), y ahora sí, en la bandeja de Ana llegó el aviso de que su transferencia no se realizó y se le devolvió el dinero, que hasta la S08 nadie le daba.



Figura 10: Al volver `notificacion`: la cola queda en 0, el consumidor procesa las transferencias 3, 4 y 5, y llegan los avisos a las dos bandejas.

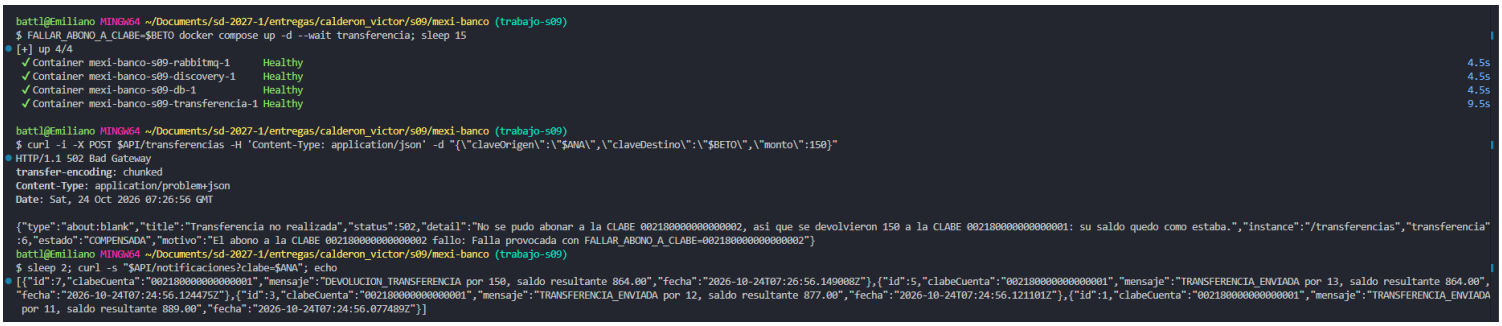


Figura 11: La transferencia 6 responde `502` con estado `COMPENSADA`, y a la bandeja de Ana llega `DEVOLUCION_TRANSFERENCIA por 150`.

3.4 · ¿Por qué el ack va después de guardar?

Si `notificacion` muere justo entre las dos líneas, con este orden (primero guardar y luego el ack) los avisos ya están en la base pero el broker nunca recibió el ack, por lo que vuelve a entregar el mensaje y el aviso *se repite*. Con el orden al revés (primero el ack y luego guardar), el broker ya borró el mensaje cuando el proceso muere sin haber guardado nada, por lo que el aviso *se pierde* para siempre. Entre repetir y perder, repetir se puede arreglar (con idempotencia, que es la siguiente parte), y perder no.

4. El duplicado

Hora: 01:29 (antes) y 01:35 (después)

Tal como lo vimos en la clase, la mensajería entrega *al menos una vez* (at-least-once), y justo por el orden que acabo de explicar (primero guardar, luego el ack), si `notificacion` muere en medio, el mensaje se vuelve a entregar. Para provocarlo elegí la forma de matar al consumidor: agregué en `ConsumidorDeTransferencias` una pausa configurable entre guardar y hacer el ack (`PAUSA_ANTES_DEL_ACK_MS`, que también agregué al `environment` de `notificacion` en el compose y que por omisión vale 0), levanté `notificacion` con una pausa de 15 segundos e hice una transferencia de 7. En una sola línea, dos segundos después revisé la cola y el log, y maté al consumidor con `docker compose kill` (es decir, sin darle oportunidad de terminar):

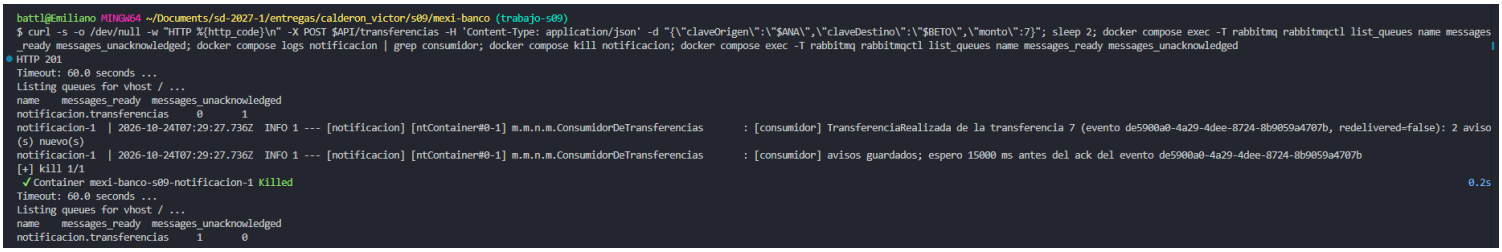


Figura 12: Durante la pausa, la cola tiene 1 mensaje *unacked* y el log dice que los avisos ya se guardaron; al matar `notificacion`, el mensaje regresa a *ready*.

Ahí se ve exactamente el problema: durante la pausa la cola tenía un mensaje *unacked*, es decir, el broker ya lo había entregado pero no sabía si se había procesado, mientras que los avisos de la transferencia 7 ya estaban guardados en la base. Al matar al consumidor, el broker vio que la conexión se cerró sin el ack y regresó el mensaje a *ready*. Después volví a levantar `notificacion` sin la pausa:



Figura 13: El antes: al volver, el mismo evento llega con `redelivered=true`, se guardan otros 2 avisos y Beto tiene `TRANSFERENCIA_RECIBIDA por 7` dos veces.

El mismo evento (`de5900a0-...`) llegó otra vez con `redelivered=true`, y como el consumidor no tenía forma de saber que ya lo había procesado, volvió a guardar los dos avisos, por lo que en la bandeja de Beto quedó dos veces el aviso de que recibió 7 (y lo mismo en la de Ana con el envío), aunque la transferencia fue una sola.

4.1 · El arreglo: idempotencia por `eventoId`

Para arreglarlo, lo que hice fue guardar el `eventoId` de cada evento procesado en una tabla nueva, `evento_procesado`, donde el `eventoId` es la llave primaria (es decir, la base no deja que haya dos filas con el mismo), y lo más importante es que se guarda *en la misma transacción* que los avisos. La marca la escribo con una sola sentencia de Postgres en `EventoProcesadoRepository`:

```
@Modifying
@Query(nativeQuery = true, value = """
    INSERT INTO evento_procesado (evento_id, tipo, transferencia_id, procesado)
    VALUES (:eventoId, :tipo, :transferenciaId, now())
    ON CONFLICT (evento_id) DO NOTHING""")
int marcarSiEsNuevo(UUID eventoId, String tipo, Long transferenciaId);   // 1 = nuevo, 0 = repetido
```

Y al principio de `avisarDeTransferencia`, dentro del mismo `@Transactional` que el `saveAll` de los avisos:

```
if (procesados.marcarSiEsNuevo(evento.eventoId(), evento.tipo(), evento.transferenciaId()) == 0) {
    log.info("[notificacion] el evento {} ya se habia procesado: no se guarda ningun aviso", ...);
    return List.of();   // 0 avisos nuevos, y el consumidor hace el ack igual
```

```
}
```

Algo que tuve en cuenta es que lo hice con una sola sentencia y no como "consultar si existe y luego insertar", ya que entre esas dos llamadas otra entrega del mismo mensaje se podría colar y las dos verían que no existe. Tampoco dejé que el `INSERT` tronara por la llave duplicada para atrapar la excepción, porque en Postgres eso deja la transacción abortada y ya no se puede hacer nada más en ella; con `ON CONFLICT DO NOTHING` la base decide sola y sin error, y me regresa cuántas filas insertó. Que la marca vaya en la misma transacción que los avisos es lo que hace que funcione: si algo falla después de la marca, el *rollback* se lleva la marca y los avisos juntos, y la siguiente entrega lo intenta desde cero; y si el commit sí pasó, ya existen los dos, por lo que una reentrega encuentra la marca y no guarda nada. Y el consumidor hace el ack igual en el caso del repetido, porque si no lo confirmara, el broker se lo seguiría entregando para siempre.

4.2 · El experimento otra vez

Con el arreglo desplegado repetí exactamente lo mismo, ahora con una transferencia de 8:

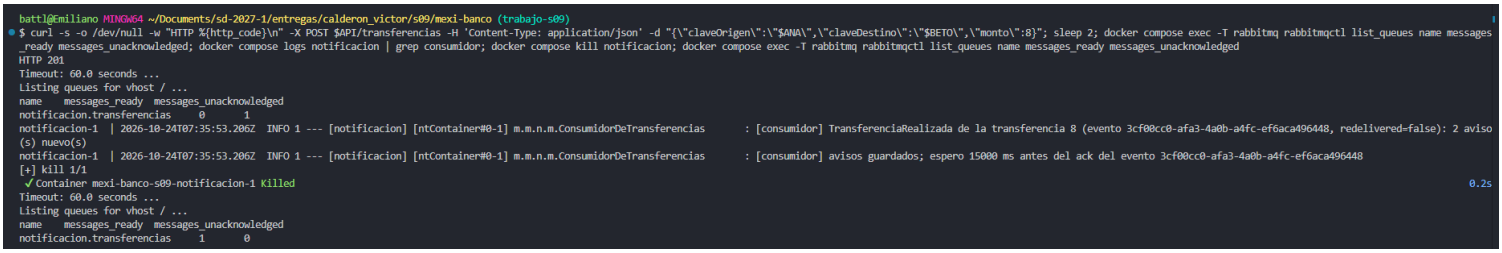


Figura 14: El mismo experimento con el arreglo: 1 *unacked* durante la pausa y de regreso a *ready* al matar `notificacion`.



Figura 15: El después: la segunda entrega llega con `redelivered=true`, pero el evento ya estaba marcado y se guardan 0 avisos nuevos; Beto tiene un solo aviso por 8.

La primera parte se comportó igual que antes: el consumidor guardó los avisos de la transferencia 8, se quedó en la pausa con el mensaje *unacked*, lo maté y el mensaje regresó a *ready*. La diferencia está al volver: el mismo evento (`3cf00cc0-...`) llegó otra vez con `redelivered=true`, pero ahora `NotificacionService` encontró su marca y escribió en el log que ya se había procesado, el consumidor reportó `0 aviso(s) nuevo(s)` e hizo el ack. En la bandeja de Beto el aviso `TRANSFERENCIA_RECIBIDA por 8` aparece una sola vez (mientras que el de 7, del experimento anterior, sigue repetido), y en la tabla `evento_procesado` está la fila del evento de la transferencia 8. Es decir, el mensaje se entregó

dos veces, como lo permite el “al menos una vez”, pero el efecto quedó una sola vez, que es lo que en la clase llamamos un consumidor idempotente.

5. El mensaje envenenado y la DLQ

Hora: 01:38 (antes) y 01:45 (después)

Un mensaje envenenado (*poison pill*) es uno que nunca se va a poder procesar, por más que se reintente. Para ver qué hacía mi consumidor con uno de esos, todavía con la versión de la sección 4, publiqué desde el panel, en el exchange `transferencias` y con la clave `transferencia.realizada`, un cuerpo que ni siquiera es un JSON completo:



Figura 16: Publicando `{"hola":` desde el panel, en el exchange `transferencias` con la clave `transferencia.realizada`.

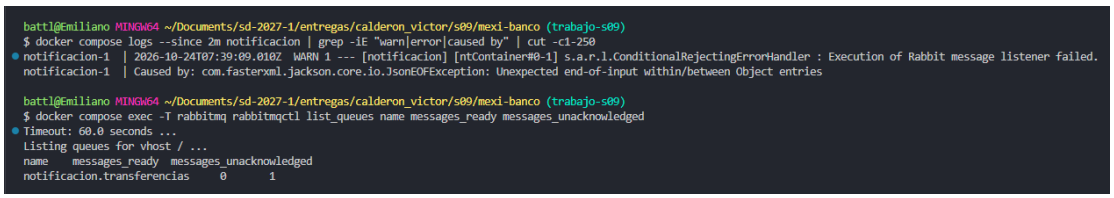


Figura 17: El antes: el consumidor truena con `JsonEOFException`, Spring solo lo reporta como `WARN` y el mensaje se queda *unacked*.

Lo que pasó es que `json.readValue` lanzó una `JsonEOFException` (se acabó el texto a la mitad de un objeto), la excepción salió del `@RabbitListener` y el manejador de errores de Spring (`ConditionalRejectingErrorHandler`) solo la escribió en el log como `WARN`. Pero como el ack es manual, nadie le respondió nada al broker, ni ack ni nack, por lo que el mensaje no rebotó ni tiró al consumidor: se quedó trabado en *unacked*, como se ve también en el panel:

Después hice una transferencia normal de 9 para ver si el veneno estorbaba:

La transferencia 9 sí generó sus dos avisos, ya que con `prefetch: 5` el consumidor puede tener hasta cinco mensajes sin confirmar a la vez, y el veneno solo ocupa uno de esos lugares. Sin embargo, sigue ahí: si `notificacion` se reinicia, el broker lo vuelve a entregar y se vuelve a trabar, y si llegaran

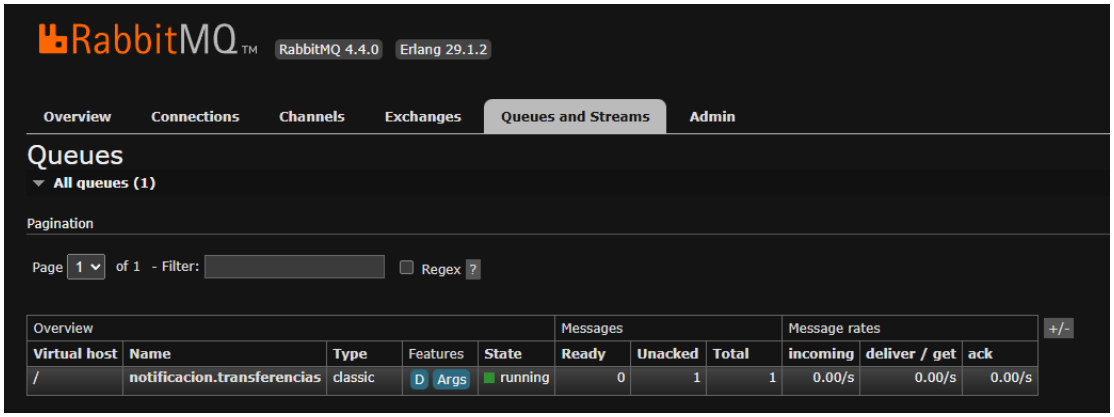


Figura 18: La cola con el mensaje envenenado trabado: 0 *ready* y 1 *unacked*.

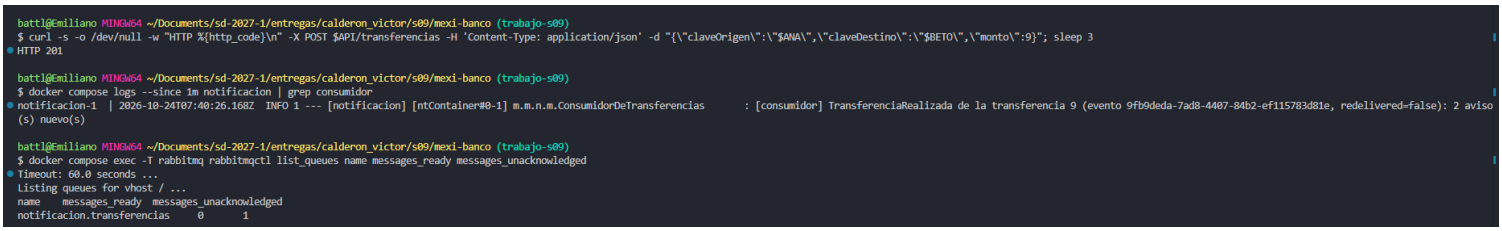


Figura 19: Una transferencia normal después del veneno sí genera sus avisos, pero el mensaje malo sigue *unacked*.

cinco mensajes así, el consumidor se quedaría sin lugares y dejaría de procesar *todo*, aunque los demás mensajes estuvieran bien.

5.1 · El arreglo: DLX y DLQ

La solución es apartar el mensaje que no se puede procesar en una *dead letter queue*, donde no estorbe y una persona lo pueda revisar. Son tres piezas, con los nombres del contrato, y las declaré en el mismo `Declarables` de `notificacion`:

```
Queue cola = QueueBuilder.durable("notificacion.transferencias")
    .deadLetterExchange("transferencias.dlx")                  // x-dead-letter-exchange
    .deadLetterRoutingKey("notificacion.transferencias")       // x-dead-letter-routing-key
    .build();
DirectExchange dlx = ExchangeBuilder.directExchange("transferencias.dlx").durable(true).build();
Queue dlq = QueueBuilder.durable("notificacion.transferencias.dlq").build();
... BindingBuilder.bind(dlq).to(dlx).with("notificacion.transferencias")
```

Es decir, la cola principal tiene dos argumentos que le dicen al *broker* a dónde mandar un mensaje que el consumidor rechaza sin reencolar, y es el broker quien lo mueve, no mi servicio. En el consumidor separé la lectura en un método `leer` que lanza `EventoIlegibleException`, y cada mensaje ahora termina de una de tres formas:

```
try {
    evento = leer(mensaje.getBody());
} catch (EventoIlegibleException veneno) {
    canal.basicNack(etiqueta, false, false);   // el último false es requeue: al DLX, no a la cola
    return;
}
try {
```

```
    avisos = notificaciones.avisarDeTransferencia(evento);
} catch (RuntimeException fallaNuestra) {      // la base, por ejemplo: el mensaje está bien
    esperar(esperaAntesDeReencolarMs);
    canal.basicNack(etiqueta, false, true);    // este sí se reencola
    return;
}
canal.basicAck(etiqueta, false);
```

Para pensar la pregunta de la guía: un JSON bien formado pero con un tipo que no existe, o sin `eventoId`, también lo considero veneno, porque tampoco se va a arreglar por más veces que se reintente (y sin `eventoId` ni siquiera podría aplicar la idempotencia). Por eso `leer` no solo atrapa el error de Jackson, sino que también revisa que el tipo sea `TransferenciaRealizada` o `TransferenciaRevertida` y que vengan los campos que el contrato pide. En cambio, si lo que falló fue mi base de datos, el mensaje está bien y el problema es mío, ahí sí conviene reencolarlo; le puse una pausa de 2 segundos antes del nack para no darle vueltas en caliente mientras la base vuelve.

5.2 · El `PRECONDITION_FAILED` (y un mensaje que se perdió)

Al desplegar esta versión me encontré con el error que advertía la guía, pero con un detalle que no esperaba:

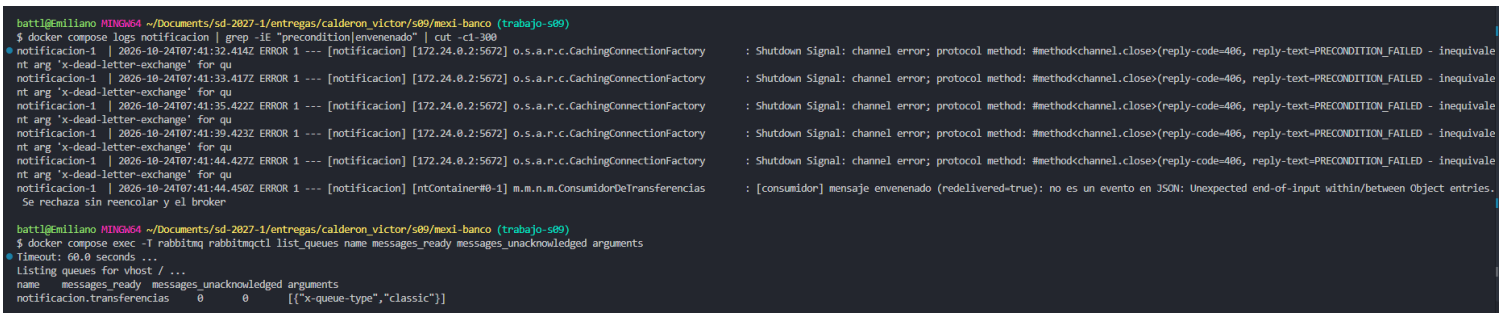


Figura 20: Al arrancar con la cola vieja: `PRECONDITION_FAILED` por el argumento `x-dead-letter-exchange`, el consumidor arranca de todos modos y el veneno que estaba trabado se rechaza… hacia ningún lado.

RabbitMQ no deja cambiar los argumentos de una cola que ya existe, por lo que la declaración falló con `PRECONDITION_FAILED - inequivalent arg 'x-dead-letter-exchange'`. Lo que no me esperaba es que eso no tumbó al servicio: Spring siguió reintentando la declaración (y escribiendo el error cada pocos segundos), terminó de arrancar y el consumidor se conectó a la cola *vieja*, la que no tiene DLX. Lo primero que recibió fue el mensaje envenenado que estaba trabado (que regresó a *ready* al apagarse el contenedor anterior), lo rechazó sin reencolar como debía, pero como esa cola no tenía a dónde mandar sus rechazados, el broker simplemente lo *tiró*. Además, como la declaración se cayó justo en la cola principal, la DLQ ni siquiera se alcanzó a crear. Es decir, ese mensaje se perdió, y justo por esto la guía pide borrar la cola *antes* de reiniciar el servicio.

5.3 · La DLQ funcionando

Ya con la cola vacía, la borré desde el panel (*Queues and Streams*, la cola, *Delete*) y reinicié `notificacion`. Esta vez la declaración pasó completa: la cola principal ya tiene las marcas `DLX` y `DLK` (sus dos argumentos nuevos) y apareció la DLQ:

Después volví a publicar `{"hola":` desde el panel, igual que en el "antes":

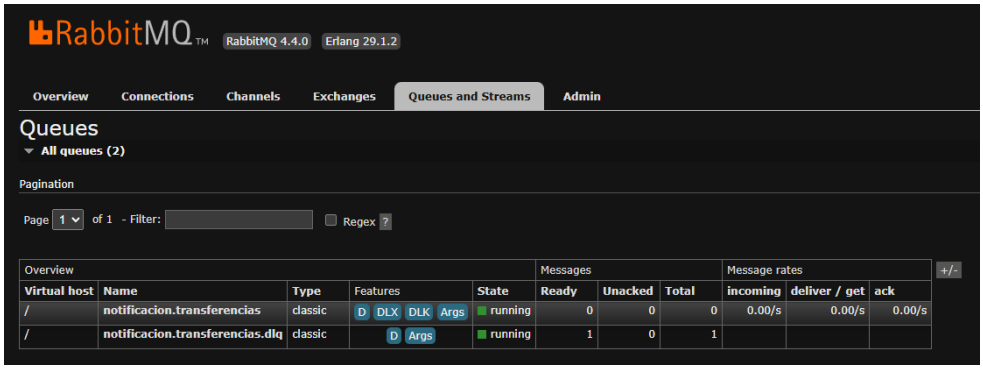


Figura 21: Después de borrar la cola y reiniciar: `notificacion.transferencias` con `DLX` y `DLK`, y la nueva `notificacion.transferencias.dlq`.

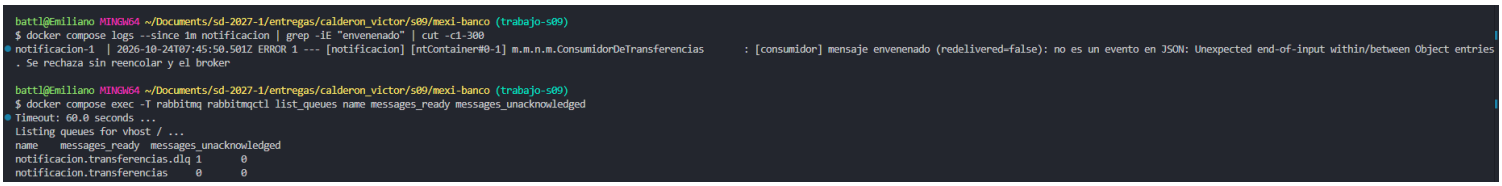


Figura 22: El después: el consumidor detecta el veneno, lo rechaza sin reencolar y queda 1 mensaje en la DLQ; la cola principal queda en 0.

Ahora el consumidor lo reconoció como mensaje envenenado, lo rechazó sin reencolar, y el broker lo movió a `notificacion.transferencias.dlq`, por lo que la cola principal quedó limpia (0 *ready* y 0 *unacked*). Al revisarlo en la DLQ con *Get messages* (con *Nack message requeue true*, para verlo sin sacarlo de ahí) se ve que llegó por `transferencias.dlx`, con su cuerpo original y con la cabecera `x-death`, que dice de qué cola salió, con qué clave había llegado y por qué: `reason: rejected`.

Por último, una transferencia normal de 10 para comprobar que todo lo demás sigue funcionando:

La transferencia 10 generó sus dos avisos, y el veneno se quedó apartado en la DLQ sin ocupar ningún lugar del *prefetch* del consumidor, esperando a que una persona lo revise.

6. Preguntas de cierre

1. Si RabbitMQ está caído cuando la saga llega a `COMPLETADA`, ¿qué pasa con el aviso en tu versión? ¿Por qué no lo arregla el *publisher confirm*?

En la versión de las partes 1 a 4 el aviso se pierde: la transferencia ya guardó `COMPLETADA` en su base, después intenta publicar, la conexión al broker se rinde a los 2 segundos (`connection-timeout`), `avisar` lo escribe en el log y sigue, por lo que el cliente recibe su `201` pero nadie vuelve a intentar mandar ese evento, ni siquiera cuando RabbitMQ regresa (lo comprobé en la sección 7). El *publisher confirm* no lo arregla porque lo único que hace es decirme si el broker recibió o no el mensaje, es decir, me sirve para *enterarme* de que no se publicó, pero no guarda el evento en ningún lado para reintentarlo; y con el broker caído ni siquiera hay alguien que confirme. El problema de fondo es la doble escritura: la base y el broker son dos sistemas sin una transacción que los una, y en este caso la base ya se escribió y el broker no. En mi versión final, con el outbox del reto, esto ya no pasa, porque el evento se guarda en la misma transacción que el `COMPLETADA` y el relay lo publica en cuanto el broker vuelve.

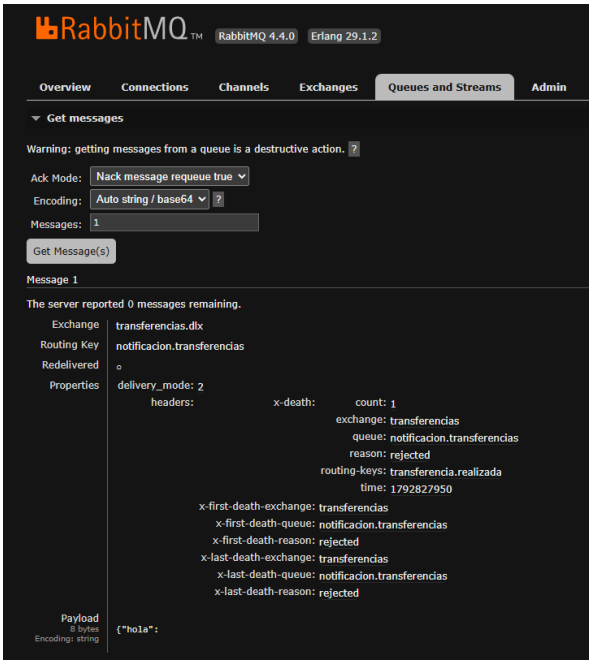


Figura 23: El mensaje en la DLQ: llegó por `transferencias.dlx` y su cabecera `x-death` dice que se rechazó (`rejected`) en `notificacion.transferencias`.

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s09/mexi-banco (trabajo-s09)
$ curl -s -o /dev/null -w "HTTP %{http_code}\n" -X POST $API/transferencias -H 'Content-Type: application/json' -d "{\"claveOrigen\":\"$ANA\",\"claveDestino\":\"$BETO\",\"monto\":10}"; sleep 3
HTTP 201

battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s09/mexi-banco (trabajo-s09)
$ docker compose logs --since 1m notificacion | grep consumidor
notificacion-1  | 2026-10-24T07:48:13.858Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : [consumidor] TransferenciaRealizada de la transferencia 10 (evento c6162efd-017c-4a5e-92b2-c4cbe88a61fe, redelivered=false): 2 avis
o(s) nuevo(s)

battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s09/mexi-banco (trabajo-s09)
$ curl -s "$API/notificaciones?clabe=$BETO"; echo
[{"id":17,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 10, saldo resultante 670.00","fecha":"2026-10-24T07:48:13.826774Z"},{"id":15,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 9, saldo resultante 660.00","
fecha":"2026-10-24T07:40:26.140299Z"},{"id":13,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 8, saldo resultante 651.00","fecha":"2026-10-24T07:35:53.177024Z"},{"id":11,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBI
DA por 7, saldo resultante 643.00","fecha":"2026-10-24T07:30:17.613813Z"},{"id":9,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 7, saldo resultante 643.00","fecha":"2026-10-24T07:29:27.693050Z"},{"id":6,"clabeCuenta":"0021800000000000
2","mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-24T07:24:56.124627Z"},{"id":4,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00","fecha":"2026-10-24T07:24:56.121127Z"},{"id
":2,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00","fecha":"2026-10-24T07:24:56.077549Z"}]

battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s09/mexi-banco (trabajo-s09)
$ docker compose exec -T rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias.dlq 1       0
notificacion.transferencias     0       0
```

Figura 24: Las transferencias normales siguen generando avisos: la transferencia 10 llega a la bandeja de Beto y el veneno se queda apartado en la DLQ.

2. ¿Por qué el ack va después de guardar y no antes? ¿Qué garantía obtienes con cada orden?

Porque el ack es lo que le dice al broker que ya puede borrar el mensaje, por lo que solo debo darlo cuando lo que el mensaje pedía ya está a salvo en mi base. Con el ack *después* de guardar obtengo *al menos una vez*: si el consumidor muere entre el commit y el ack, el mensaje se vuelve a entregar y el aviso se puede repetir, pero nunca se pierde (y la repetición la resolví con la idempotencia de la sección 4). Con el ack *antes* de guardar obtendría *a lo más una vez*: si el consumidor muere entre el ack y el commit, el broker ya lo borró y el aviso se pierde para siempre, sin que nadie se entere. Para un banco, un aviso repetido se puede detectar y descartar, pero uno perdido no se puede recuperar, por eso conviene el primer orden.

3. Elige y di qué sacrificaste.

Elección: Avisar con un evento en RabbitMQ en lugar de una llamada HTTP síncrona a `notificacion`.

Porque: Un aviso, que puede esperar, ya no tumba una transferencia que ya movió el dinero, y `transferencia` y `notificacion` ya no tienen que estar vivos al mismo tiempo.

Sacrifico: La certeza inmediata: cuando el cliente recibe su `201`, el aviso todavía no existe y `transferencia` ya no sabe si llegó (consistencia eventual); a cambio cargo con duplicados y mensajes envenenados que antes no existían, y con un componente más (el broker) que también se puede caer.

7. Reto: Transactional Outbox

Hora: 01:55

Hasta aquí, `notificacion` se puede caer y no se pierde nada, pero falta el otro lado: ¿qué pasa si el que está caído es RabbitMQ cuando la saga llega a `COMPLETADA`? Primero lo probé con la versión de las partes anteriores, apagando el broker (`docker compose stop rabbitmq`) y mandando una transferencia de 1:

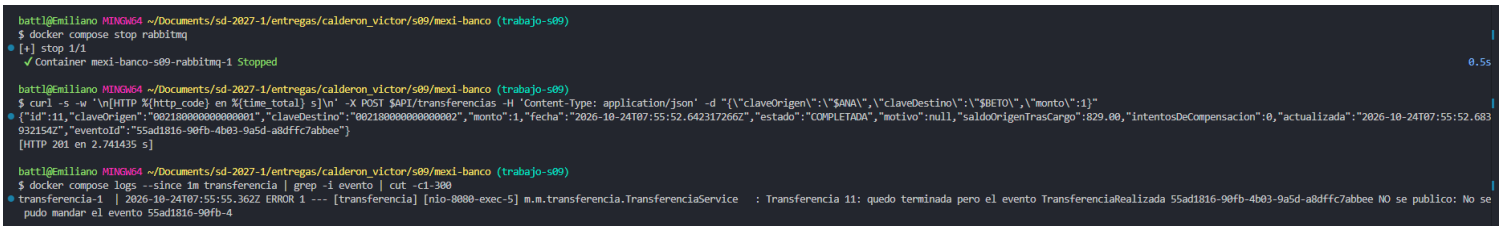


Figura 25: Sin outbox y con RabbitMQ apagado: la transferencia 11 responde `201` y queda `COMPLETADA` (en 2.7 s), pero su evento no se publica.

La transferencia respondió `201` y quedó `COMPLETADA`, aunque tardó 2.7 segundos (los 2 del `connection-timeout` intentando llegar al broker), y en el log quedó que el evento no se pudo mandar. Después volví a encender RabbitMQ y esperé:

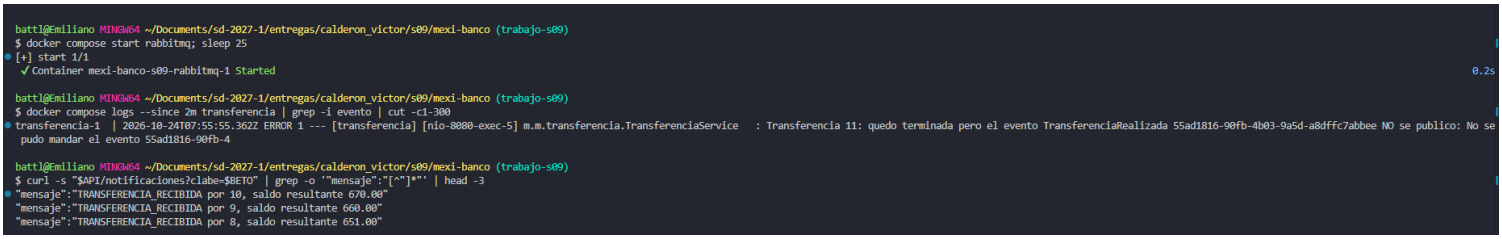


Figura 26: Al volver RabbitMQ nadie vuelve a mandar el evento: en el log solo está el error y en la bandeja de Beto no aparece el aviso por 1.

Y efectivamente, al volver el broker nadie mandó ese evento: en el log solo está el error de antes y en la bandeja de Beto el último aviso sigue siendo el de 10, es decir, a Beto nunca se le va a avisar que recibió ese peso. Es la doble escritura que vimos en la clase: la base dice que la transferencia se completó y el evento nunca sale, porque la base y el broker son dos sistemas distintos sin una transacción que los una.

7.1 · El outbox

La idea del Transactional Outbox es no escribir en dos sistemas a la vez: la saga ya no habla con RabbitMQ, sino que deja el evento en *su propia* base, en una tabla `outbox`, y lo hace en la *misma transacción local* que el estado final de la transferencia, por lo que o quedan los dos o ninguno. Después, un *relay* se encarga de llevar lo pendiente al broker. Los cambios quedaron así (todo en `transferencia`):

- `EventoPendiente`: la fila de la tabla `outbox`, con el `eventoId` (único), el tipo, la transferencia, el evento en JSON, cuándo se creó, cuándo se publicó (`null` mientras esté pendiente), cuántos intentos lleva y el último error.
- En `TransferenciaService`, el método `avisar` se fue y en su lugar está `guardarConEvento`, que guarda la transferencia y su evento dentro de un `TransactionTemplate`. Usé el *template* y no `@Transactional` porque el método es privado y se llama desde la misma clase, y en ese caso la anotación no pasa por el proxy de Spring y no hace nada:

```
Transferencia guardada = transaccion.execute(estado -> {
    Transferencia g = transferencias.save(t);     // COMPLETADA o COMPENSADA
    outbox.guardar(evento.apply(g));               // y su evento, en la MISMA transacción
    return g;
});
```

  Además, `Outbox.guardar` lleva `@Transactional(propagation = MANDATORY)`, es decir, si alguien lo llamara fuera de una transacción, truena en lugar de guardar el evento por su lado (que sería otra vez una doble escritura).

- `RelayDelOutbox`: un `@Scheduled` que cada segundo llama a `Outbox.publicarPendientes()`. Esa vuelta, en una transacción, toma hasta 20 filas pendientes con `SELECT ... FOR UPDATE SKIP LOCKED`, las publica en orden con el mismo `PublicadorDeEventos` de la Parte 1 (que espera la confirmación del broker) y marca cada una como publicada. En cuanto una no se puede publicar, anota el error y deja las demás para la siguiente vuelta, ya que si el broker no contesta no tiene caso esperar su *timeout* una vez por evento. El `SKIP LOCKED` es para cuando hay más de una réplica de `transferencia`: cada relay se lleva filas distintas en lugar de publicar las mismas dos veces.

Algo que también tuve que pensar es el *health check*: el *starter* de AMQP agrega el estado del broker a `/actuator/health`, por lo que con RabbitMQ apagado, `transferencia` se marcaría `unhealthy` aunque con el outbox sigue haciendo su trabajo perfectamente. Por eso lo quité de su *health* (`management.health.rabbit.enabled: false`), ya que el broker dejó de ser una dependencia para aceptar transferencias. Como la saga ahora usa el outbox en lugar del publicador, también cambié el doble de `TransferenciaServiceTest` por uno del outbox para que siga pasando.

7.2 · Con el outbox

Desplegué esta versión de `transferencia` y repetí el experimento, ahora con una transferencia de 2:

Esta vez la transferencia respondió `201` en 0.2 segundos, ya que la saga ni siquiera intenta hablar con el broker: solo guardó la fila `COMPLETADA` y su evento en la tabla `outbox`. Mientras tanto, el relay intentaba publicarlo cada segundo y fallaba (en el log se ven los intentos 3 a 6), y en la tabla el evento seguía con `publicado` vacío y los intentos subiendo. Al encender RabbitMQ:

Después de 33 intentos fallidos, en cuanto el broker volvió el relay publicó el evento sin que yo hiciera nada, la fila quedó marcada con la hora en que se publicó, y a la bandeja de Beto llegó el aviso de que recibió 2. Es decir, con RabbitMQ apagado la transferencia se completó, y al encenderlo el evento salió



Figura 27: Con outbox y RabbitMQ apagado: la transferencia 12 responde `201` en 0.2 s y su evento queda pendiente en la tabla `outbox`, mientras el relay lo reintenta cada segundo.

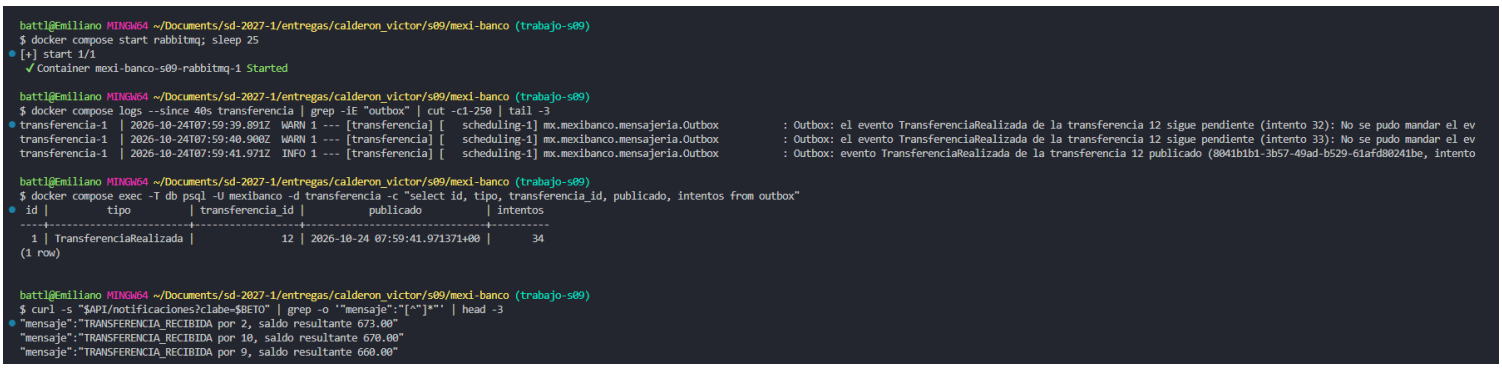


Figura 28: Al volver RabbitMQ, el relay publica el evento solo (en el intento 34), la fila queda con su fecha de `publicado` y a Beto le llega el aviso por 2.

solo y llegó el aviso, que es justo lo que pedía el reto. Algo que también se nota es que, con el broker caído, la transferencia respondió mucho más rápido que sin el outbox (0.2 contra 2.7 segundos), porque el tiempo de esperar al broker ya no lo paga el cliente, sino el relay en segundo plano.

¿Quién me salva si el relay publica y muere antes de marcar la fila? El evento vuelve a salir en la siguiente vuelta, pero sale con el mismo `eventoId` (se guardó en la fila del outbox), por lo que el consumidor idempotente de la sección 4 lo reconoce y no guarda avisos otra vez. Es decir, el outbox me da "al menos una vez" del lado del productor, y la idempotencia del consumidor es la que lo convierte en "un solo juego de avisos".

Entrega

En mi carpeta `entregas/calderon_victor/s09/` quedan esta bitácora y la carpeta `codigo/`, con los dos servicios completos (sin `target/`) y el `docker-compose.yml`, que sí cambié:

- `codigo/transferencia/`: el publicador con *publisher confirms*, el evento con su `eventoId`, la saga sin `NotificacionCliente` y el outbox con su relay.
- `codigo/notificacion/`: la cola con su DLX y DLQ, el consumidor con ack manual que manda el veneno a la DLQ, y la idempotencia por `eventoId` (tabla `evento_procesado`).
- `codigo/docker-compose.yml`: el nombre del proyecto y de la red (`mexi-banco-s09`), la variable `PAUSA_ANTES_DEL_ACK_MS` de `notificacion` y sin `NOTIFICACION_URL` en `transferencia`.

Las pruebas que ya traían los dos servicios (con el ajuste de `TransferenciaServiceTest` que pide la guía) pasan; las corrí en mi máquina con `./mvnw test`, aunque tuve que agregar una opción, ya que tengo instalado Java 25 y la versión de Mockito que trae Spring Boot 3.3.4 todavía no lo reconoce (en las imágenes de Docker no hace falta, porque usan Java 21 y no compilan las pruebas):

```
./mvnw test -DargLine=-Dnet.bytebuddy.experimental=true
```